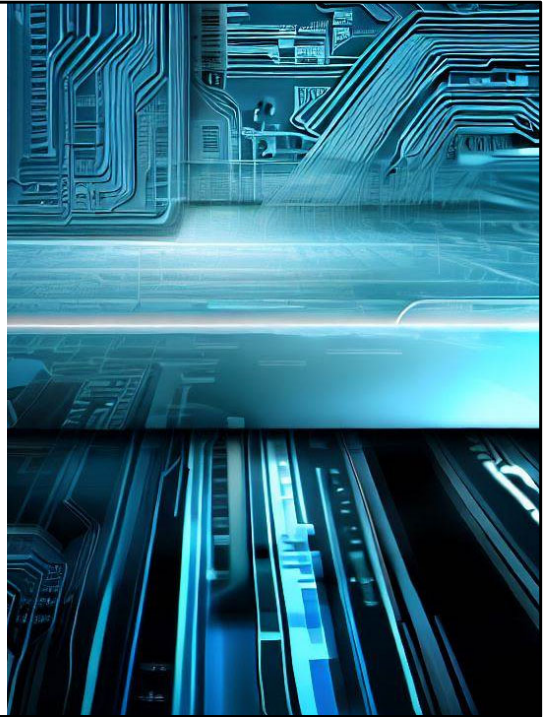


SC1006 Computer Organisation and Architecture Processor Pipeline

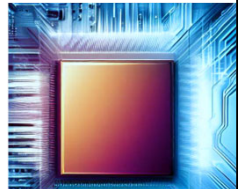
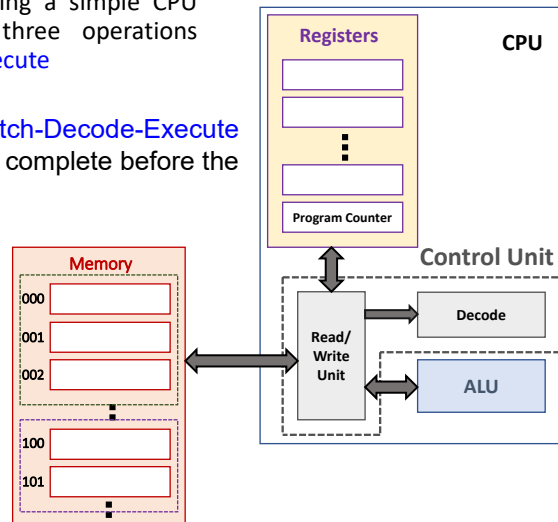
Prof Lin WeiSi
College of Computing and
Data Science, NTU
Email: wslin@ntu.edu.sg



- This chapter is on processor pipeline.
- Pipeline is a processor architecture that split an instruction into a few stages, each performing an operation that is simpler than a typical CPU instruction.
- The pipeline allows CPU to be clock at a faster rate and have a higher performance. More details in slides next.

Review of a Simple CPU

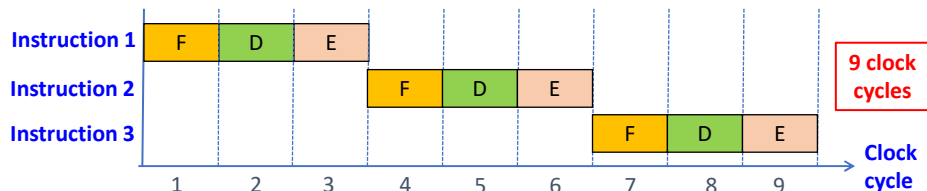
- So far we have been discussing a simple CPU which does the following three operations sequentially: **Fetch-Decode-Execute**
- In the simple CPU, the **Fetch-Decode-Execute** cycle of an instruction must complete before the next instruction is fetched



- So far, when we discuss execution of CPU instructions, we use a simple CPU architecture where the current instruction is executed completely before the next instruction is fetched from the memory to be executed.
- If we looked at the execution of a CPU instruction, we could roughly split it into 3 operations,
 - the machine code of the instruction needs to be fetched from the memory,
 - the CPU will decode the machine code to find out what instruction does for the machine code correspond to
 - After the CPU knows what the instruction is, it can proceed to perform the actual execution.
- This corresponds to the Fetch, Decode and Execute operations.
- In a simple CPU architecture, the Fetch, Decode and Execute operations of a particular instruction needs to complete before the next instruction is fetched.

Instruction Execution – Simple CPU

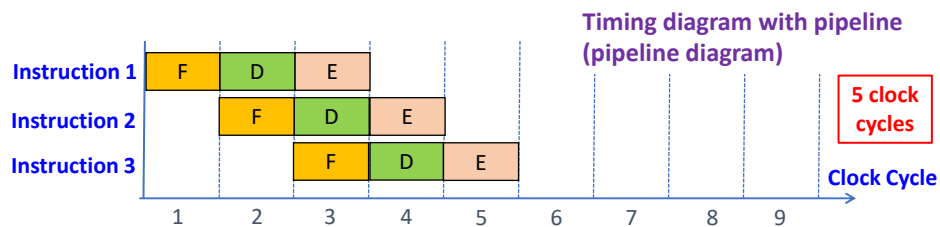
- Consider the simple CPU discussed in previous slide.
- Each instruction is broken down into the following stages
 - Fetch Instruction (F),
 - Decode (D)
 - Execute (E)
- Executing each instruction is equivalent to cycling through the three stages F, D and E. If we assume that each stage takes one clock cycle, the time taken to execute 3 instruction will be 9 cycles.



- Extending on the discussion we have in the previous slide, when a simple CPU execution an instruction, it goes through 3 stages:
 - Fetch
 - Decode
 - Execute
- If each of these stages takes one clock cycle to execute, and since the fetch for the next instruction can only be executed after completing the current instruction,
 - then three CPU instructions will take 9 clock cycles.

Pipeline Architecture

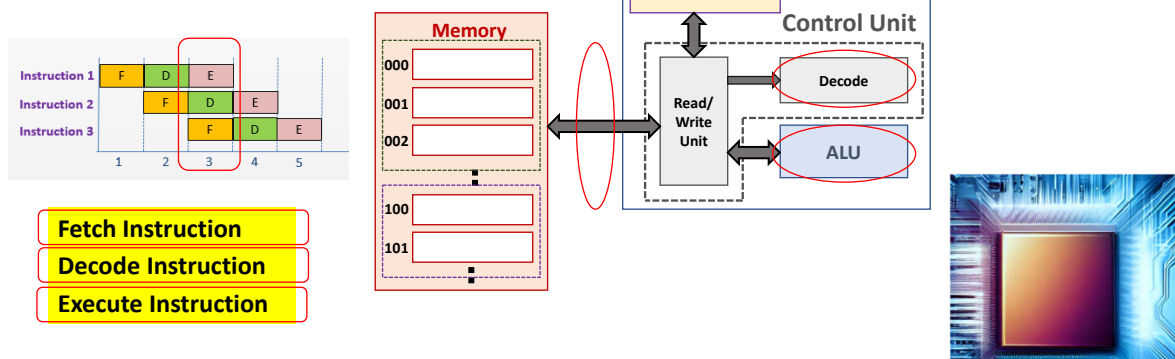
- This is an example of a Pipeline Processor Architecture.
- Pipelining is about **partitioning an instruction into simpler stages and assigning resources to allow these stages to be executed simultaneously.**
- There are many ways to partition an instruction, the example shown here is one of the simplest. Its not uncommon for more complex application processors to have more than 10 pipeline stages.



- Pipeline processors split an instruction into simpler stages and assign resources to execute these individual stages simultaneously.
- Splitting into simpler stages allows the CPU to be clock at a faster rate, since the operation at each stage is now easier.
- There are many ways to split an instructions into simpler stages. The Fetch, Decode and Execute stages discussed is only one of the ways.
 - It is not uncommon to have more than 10 pipeline stages in the more complex processors.

Pipeline – A peek into Processor Internal

- Pipelining is possible if the Fetch-Decode-Execute operations use **independent resources**
- For example
 - Fetch (External buses)
 - Decode (Instruction Decoder)
 - Execute (ALU)

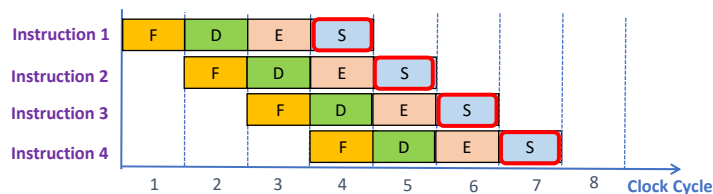


- This slide shows the processor internal operations when handling the pipeline operation.
- During the fetch stage, the processor uses the system bus to read the instructions from the memory
- The Decode stage uses the instruction decoder in the processor to decode the machine code.
- During the execute stage, the processor uses the ALU for computation.
- Notice that the hardware resources used in this example are independent from one another, i.e. none of the stages use the same hardware resource.
 - This allows all 3 stages of the pipeline to be executed simultaneously, as shown at the 3rd clock in the pipeline diagram.
- This illustrates one important criterion in order to maximise the efficiency of a pipeline, i.e. each of the pipeline stages should use independent resources; in other words, these stages should not be using the same resources.

Pipeline Efficiency



- The **efficiency** of a Pipeline architecture is **maximised** when the **pipeline is filled**
- Supposed we have a 4-stage pipeline processor
 - Instruction Fetch (F)
 - Instruction Decode (D)
 - Instruction Execution (E)
 - Data Store (S)
- It takes **4 cycles** to **fill the pipeline** and release the first instruction from the pipeline.
- But after the pipeline is filled, it is able to release **one instruction every cycle**, giving the effect of 'executing' one instruction every clock cycle.

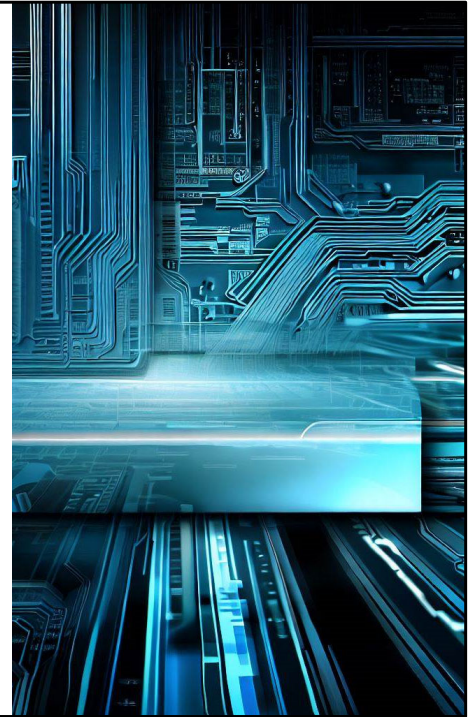


- Now, a pipeline's efficiency is maximised when the pipeline is filled, i.e. all the stages appear in a clock cycle.
- But it takes time to fill up a processor pipeline, the longer the pipe line i.e. number of stages, the longer it takes to fill it up.
- For example, in a processor with 4 pipeline stages, Fetch, Decode, Execute and Store, where
 - Fetch refers to fetching the instruction from the memory
 - Decode refers to decoding the machine code to know what instruction was fetched
 - Execute refers to performing the actual execution such as ADD, SUB etc
 - Store here refers to the storing of the results from the Execute Stage to registers or memory.
- From the pipeline diagram, we can see that it take 4 cycles to fill up the pipeline,
 - But once the pipeline is filled, the processor is able to release/finish one instruction every clock cycle.
 - This gives the effect of processor executing one instruction every clock cycle.
 - Using the above scenario, executing one instruction in a simple CPU is equivalent to executing 4 pipeline stages in a pipeline processor

above, so takes 4 times as long compared to the pipeline processor.

Pipeline Conflicts

- Pipeline efficiency mentioned in the previous slide will be reduced drastically if there are disruption to the pipeline.
- Events that disrupt the pipeline is known as pipeline conflicts.
- Pipeline conflicts typically cause a temporary halt to the pipeline or in some cases, the processor has to flush the instructions in the pipeline and reload with a fresh set of instructions.
- These actions result in additional time to execute a particular set of instructions.
- Since the total time taken to execute the instruction is longer, the number of instructions that can be released from the pipeline is reduced, which means the effective performance of the processor is lowered.
- In this module, we will look at three sources of pipeline conflict
 - Insufficient Resource
 - Data Dependency between instructions
 - Pipeline flushing due to Branch Instruction

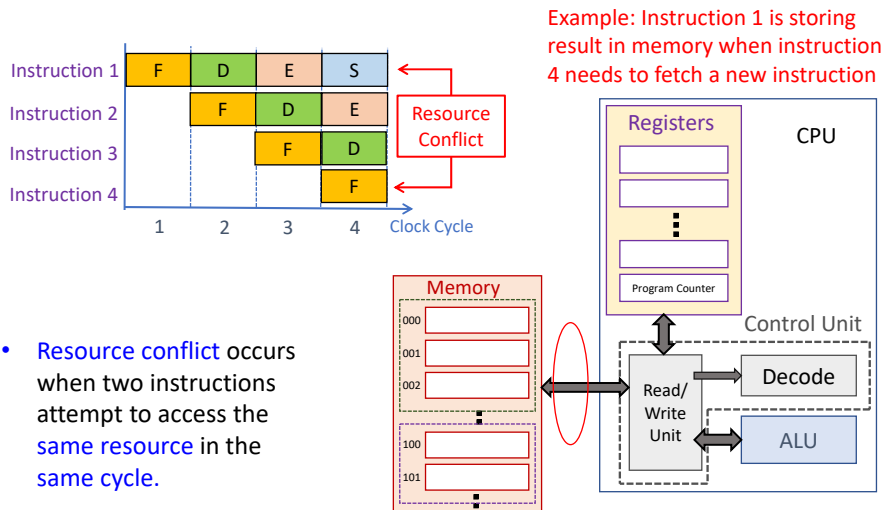


- From previous slides, we saw that pipeline efficiency is maximised when all the stages in the pipeline are filled or occupied.
- There are however, events that may disrupt the pipeline, causing
 - some stages to be halted temporarily, or
 - Some instructions to be flushed or discarded, meaning that the processor will need to refill the pipeline stages again.
- These events are known as pipeline conflicts
- The pipeline conflicts as described above will result in more time needed to execute one or more instructions.
 - Since more time is needed to execute a set of instructions, it implies that the number of instructions that can be released from the pipeline is reduced.
 - As a comparison, when the pipeline efficiency is maximised, it would be able to release one instruction every clock cycle.
- In this module, we will look into three sources of pipeline conflict
 - Conflict due to Insufficient Resource to operate each pipeline stages simultaneously
 - Conflict due to dependency of data between instructions, and this leads to additional cycles needed to mitigate the dependency.

- Additional cycles needed to execute a branch operation because some instructions that were pre-fetched into the processor has to be discarded..

Resource Conflict

- Consider a processor with 4 pipeline stages: **Fetch Instruction (F)**, **Decode (D)**, **Execute (E)**, **Store Result (S)**

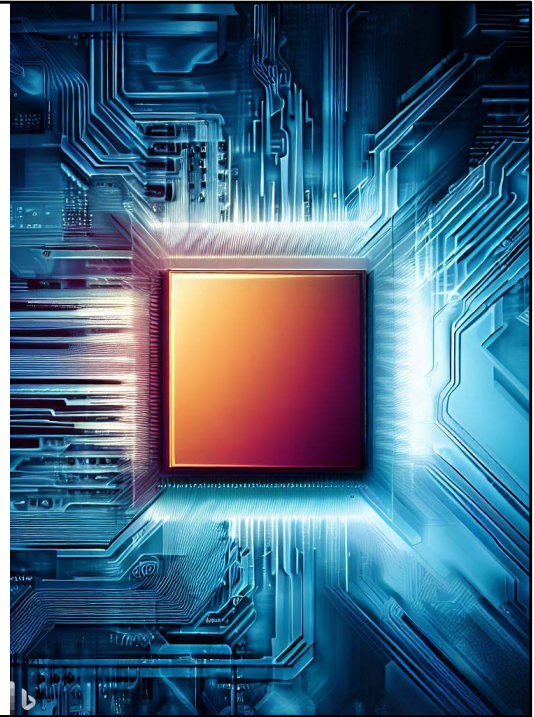


- Resource conflict** occurs when two instructions attempt to access the **same resource** in the **same cycle**.

- First, let's look at the resource conflict, which happens if the processor does not have sufficient resources to operate all the pipeline stages simultaneously.
- The diagram in the slide illustrates this point clearly.
 - Consider a 4-stage pipeline processor with Fetch, Decode, Execute, and Store stages,
 - At clock 4, the processor is fetching instruction 4 and performing a write to the memory.
 - Now instruction fetch involves reading the memory via the system bus, and
 - Store involves writing to the memory via the same system bus.
 - So if the processor only has one system bus, there will be a conflict between the two parties trying to use the system bus at clock 4.
- In summary, a resource conflict occurs when two instructions attempt to access the same resource in the same clock cycle.

Resolving Resource Conflicts

- You need sufficient resources for a pipeline processor to work efficiently
- A pipeline processor with insufficient resources to operate each pipeline stage simultaneously will result in constant halting and flushing of pipeline.
- It's common for pipeline processors to have multiple resources: internal buses (data, instruction, peripherals), control and processing units etc to allow simultaneous operations in any instance.



- There is really no other alternative in resolving resource conflict other than giving the processor what it needs to do its job.
- So you need to provide sufficient resources for a pipeline processor to work properly and efficiently.
- If the processor doesn't have the resources it needs to operate all the stage simultaneously, then at some point in time, one pipeline stage will have to wait for another and that will disrupt the pipeline operation.
- It is common for pipeline processors to have multiple resources of the same type.
E.g.
 - the processor may have multiple internal buses for transferring address, data and instruction information.
 - It may also have multiple processing units to allowing different arithmetic operations to be carried out simultaneously.

Use of ARM instructions in Pipeline Examples

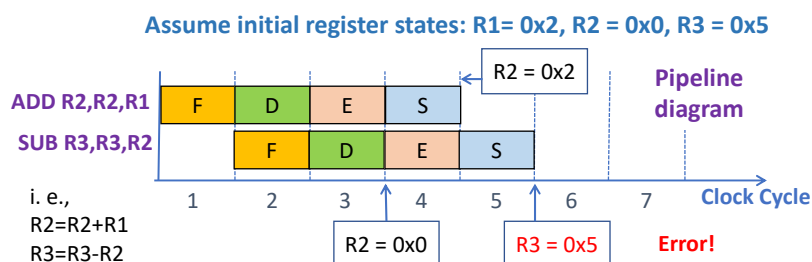
- ARM instructions are used in the Pipeline examples but note that the following key differences
 - The pipeline structure discussed is **NOT** that of the ARM pipeline
 - For simplification of analysis, **ALL** the ARM instructions used are **assumed to occupy only one word** and **each pipeline stage take one clock cycle** to execute.
 - The suffix 'S' feature is enabled by default, i.e. all instructions will affect the conditional flag. E.g. ADD = ADDS, MOV = MOVS.



- For the pipeline and the computer arithmetic topic, I'll be using the ARM assembly instructions that you have learnt during the first half to illustrate the related concepts.
- But note that the pipeline structure discussed in this chapter is not that of the ARM processor.
- To simplify the analysis, all instructions used in the pipeline discussion are assumed to occupy only one word and each pipeline stage take one clock cycle to execute.
 - The pipeline operation will be more complicated if multi-word instructions are involved as these instructions may require multiple cycles to clear one or more of the pipeline stages.
- Lastly, the suffix 'S' of ARM instructions will not be used here, or rather you can consider it to be enabled by default. That means MOV = MOVS, ADD = ADDS etc. All instructions will affect the conditional flags.
 - Our focus in this chapter is pipeline, not ARM assembly instruction operation.

Data Dependency Conflict

- Consider a 4-stage pipeline (FDES).
- Actual operation of each instruction e.g. ADD, SUB, is done during the Execute (E) stage.
- The resultant value of the execution is transferred to the destination during the Store (S) stage.
- In the example below, the **old value** (0x0) is still in R2 when SUB instruction is in **Execute** (E) stage, so R3 will have the wrong value (0x5) instead of the correct value (0x3).

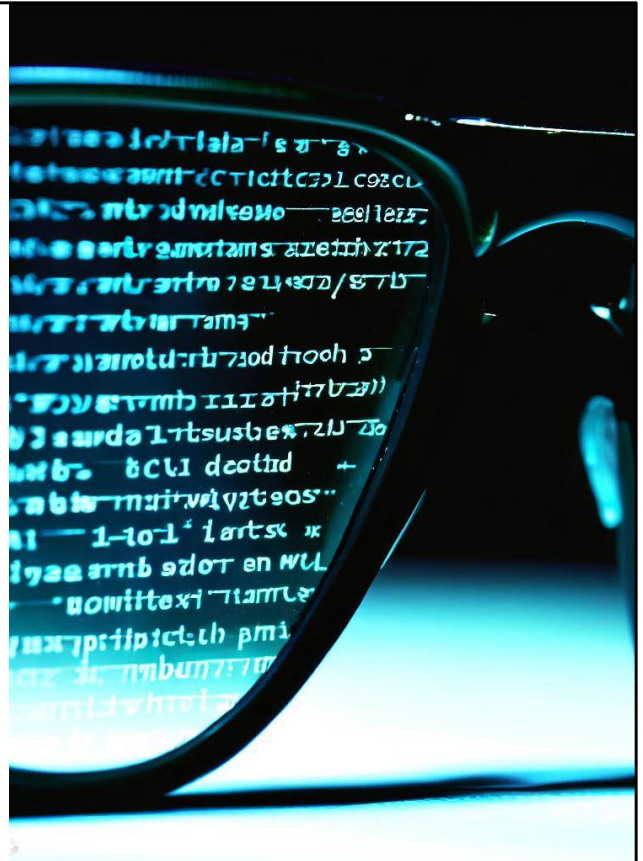


- The second conflict we will be discussing is data dependency.
 - Data dependency occurs when the current instruction has an operand that has not been updated in time for the current instruction's execution, resulting in wrong result being generated.
- I'll illustrate this with an example.
 - Consider a 4-stage pipeline processor that we discussed in previous slide, Fetch-Decode-Execute-Store.
 - The initial state of the registers are as indicated in the slide.
 - The processor is executing two instructions as shown. ADD R2, R2, R1 followed by SUB R3, R3, R2.
 - At clock cycle 3, SUB instruction is about to enter into the E stage where it will perform subtraction of R2 from R3. But notice that at this point in time, the content in R2 is still the old value, i.e. 0, instead of the updated value of 2.
 - This is because the ADD instruction will only update R2 at its S stage, which is at clock 4.
 - Therefore, the result of the SUB instruction will be wrong, it'll be 5 instead of 3.
 - This is a classic example of data dependency.

- Here, SUB instruction uses R2 as one of its operand, but R2 is the destination register of an earlier instruction.
- And because the update of R2 is not in time, the old value of R2 is used in the SUB instruction and the result is wrong.

Data Dependency Conflict

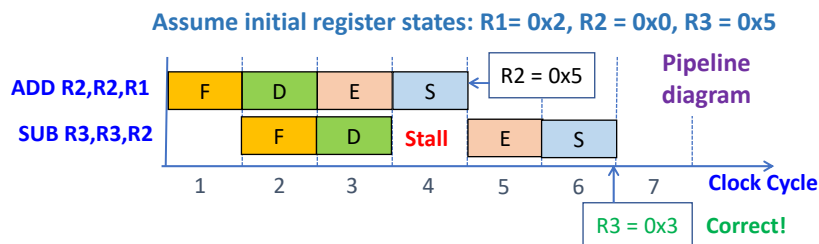
- For the example in the previous slide, it will not be an issue if the processor has a simple CPU architecture where instruction execution is sequential. The current instruction will start its execution only after the previous instruction has complete. So the most updated copy of the memory/register value is always available to the current instruction.
- Due to the overlapping nature between instructions within a pipeline, there will be instances where a data required for the proper execution of the current instruction has not been updated yet.
- This is known as data dependency issue/conflict. It arises when the source operand of the current instruction is also the destination operand of a prior instruction.
- The exact separation between the two instructions in order for the issue to occur depends on the pipeline structure and design.



- If the processor in the previous slide has a simple CPU architecture instead of a 4-stage pipeline, then data dependency issue will not occur because a simple CPU will execute ADD instruction completely, including the storage of result to R2, before it starts to execute the SUB instruction.
- But this is not the case for pipeline processor as it overlap the execution of adjacent instructions within the pipeline.
- Which means there will be scenarios where data required by current instruction has not been updated yet.
- This as discussed in previous slides, is known as data dependency.
- Note that the exact separation between two instructions in order for data dependency to occur will change depending on the pipeline structure and design.

Resolving Data Conflict – Stall the Pipeline

- **Hardware circuitry** can be used to detect data dependency between instructions
 - Compare destination identifier in the Execute Stage with source(s) in the Decode stage.

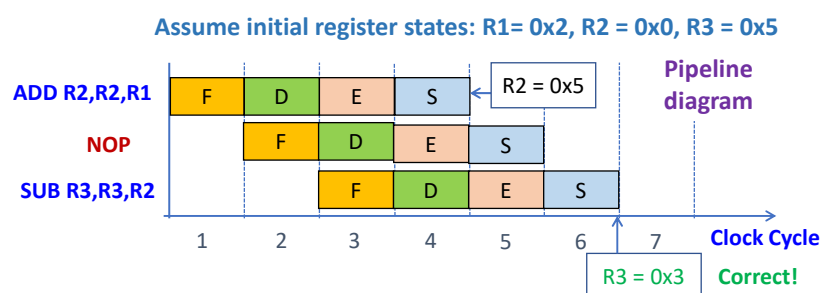


- If data dependency is detected (i.e. **R2 matches**), allow **ADD** to continue normally, but **stall** the **Decode** stage of **SUB**.
- After **ADD** completes, **SUB** is allowed to resume.

- There are two ways to resolve the data dependency.
- First method is to design the hardware to detect data dependency between instructions and stall one of the pipeline stage temporarily to allow the required data to be updated before executing the current instruction.
- Using the previous example, when hardware detected the data dependency between the **ADD** and **SUB** instructions, it will stall the **D** stage of the **SUB** instruction for one cycle to allow **R2** to be updated, then proceed with execution.

Resolving Data Conflict – Insert NOP Instructions

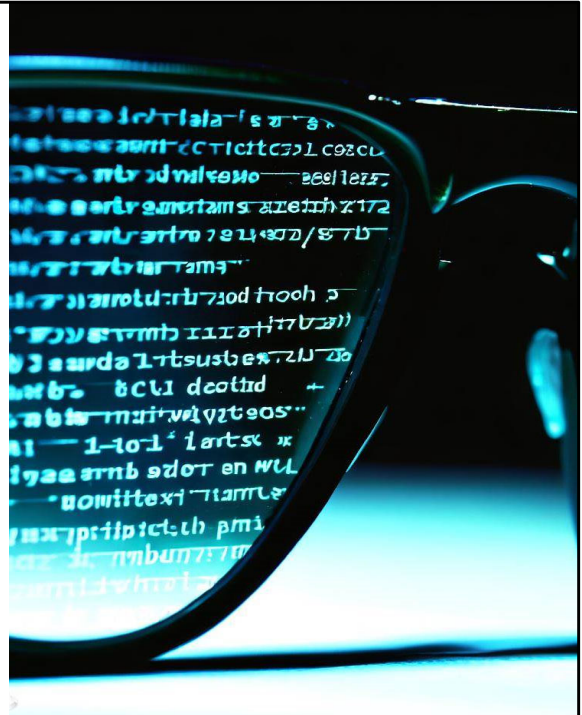
- Compiler can analyze and **insert redundant instructions** to reduce data conflict
 - Data dependencies are evident in instructions during compilation
 - Compiler inserts explicit **NOP** (No Operation) instructions **between instructions with data dependencies**
- Delay ensures new value is available in register but causes total execution time to increase



- Another way to resolve data dependency issue is to have the compiler insert additional NOP instructions between instructions with data dependency.
- This will delay the execution of the later instructions so that its operands will be the updated values.
- As shown in the diagram, the additional NOP instruction between ADD and SUB instruction causes the E stage of the SUB instruction to be delayed to clock 5, and this enable R2 to be updated to the correct value before E-stage of the SUB instruction.
- Note that NOP instruction is an assembly instruction that does nothing, and it is typically used as a place holder to delay the operation of the processor by one cycle.

Branch Instruction

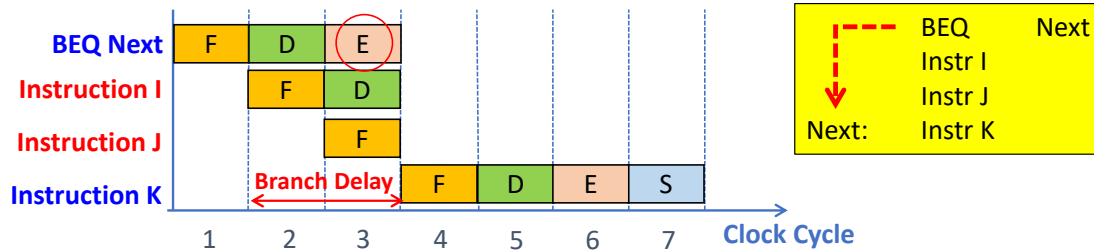
- Branch instruction usually need to perform two operations
 - Evaluate condition to determine if branch should be taken/not taken
 - If branch is taken, calculate branch target using adder in ALU
- Both operations require an ALU to perform some computation and processing so a natural stage to do this is in the Execute (E) Stage of the pipeline.
- The processor would need to flush the pipeline to reload correct instructions according to the branch decision. Flushing of pipeline is equated to wastage in cycles for instruction execution.
- The number of cycles wasted/lost is known as Branch Delay.



- Next we come to the last pipeline conflict we will be discussing for this course, the branch delay.
- A typical conditional branch instruction need to perform two operations
 - Evaluate the branch condition to decide whether the branch is to be taken or not.
 - Calculate the branch target
- Both operations require the use of an ALU to perform some computation so a natural stage in which these operations are carried out is at the E stage.
- However, due to the overlapping nature in which instructions are processed within a pipeline, unnecessary instructions may have been introduced into the pipeline before branch decision is made.
 - This means the processor will need to discard these unnecessary instructions so as to maintain proper execution of the program code.
 - Such flushing of pipeline introduce delays as some cycles are wasted, in comparison with normal cases otherwise..
- The number of cycles lost is known as the branch delay.

Branch Delay

- Branch statements in pipelining can lead to **Branch Delay**.

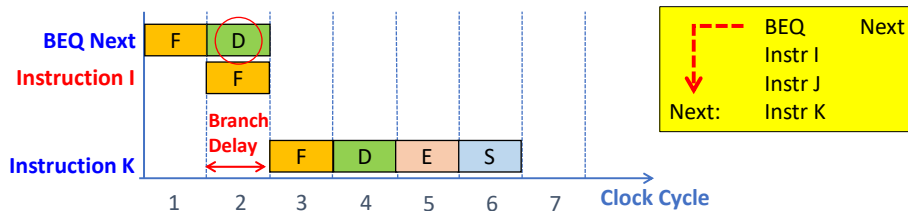


- The **branch target** is only known after the **Execute** stage, but by this time, **Instructions I and J** have already been **fetch**ed.
- Instructions I and J will be **discarded**, resulting in **two-cycle branch delay**.
- The two slots that are discarded is known as the **Delay Slots**
- Instructions I and J are known as the **Delay Slot Instructions**

- This slide illustrate the concept of branch delay discussed in the previous slide.
- Consider the code on the right, the processor pipeline will fetch these instruction sequentially into the pipeline as shown.
- At clock 3, BEQ instruction will be at its E-stage where the branch decision is made.
 - If the branch is true, then instruction K will be fetch in the next cycle as it is the next instruction to be executed after BEQ.
 - However, instructions I and J would already been fetched into the pipeline at clock 3.
 - I and J have to be discarded in order to maintain the proper execution of the program.
 - This incurred a 2-cycle branch delay, in comparison with normal pipeline cases.
- I and J are known as delay slot instructions.

Reducing Branch Delay

- Branch delay can be **reduced** by making the branch decision and calculating the branch target **earlier at the Decode stage**.
- An **additional adder** is introduced to be used in the Decode stage to enable earlier calculation of branch target.

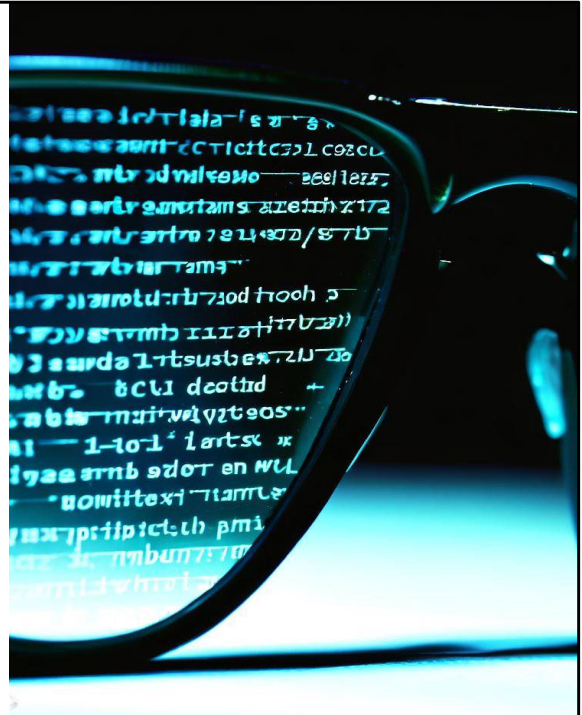


- After the **Decode stage**, the branch decision and the branch target is known.
- Hence **if branch is taken**, only **Instruction I** needs to be **discarded**.
- Branch delay is **reduced to one cycle**.

- In the previous slide, the branch decision is made at E stage and that resulted in 2-cycle branch delay.
- We can reduce the branch delay to one cycle by bringing forward the branch decision making to the D stage.
- Since performing branch decision requires an adder, one additional adder needs to be introduced into the processor to support this feature.
- With the branch decision done at D stage, we can see from the pipeline diagram that only one cycle branch delay is incurred.

Delayed Branching

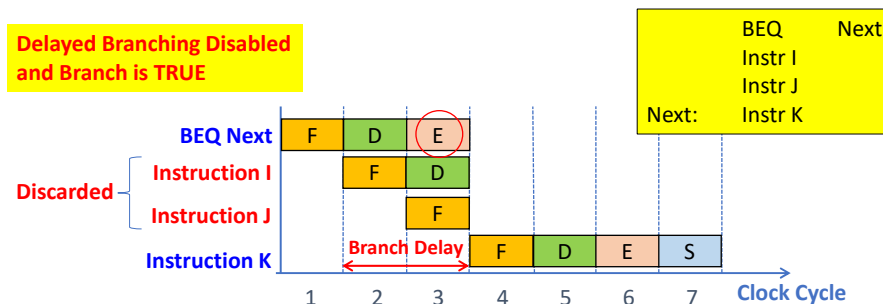
- In the previous implementation, the instruction(s) immediately following a branch is always fetched, regardless of the branch decision.
- If branch is taken, these instruction(s) will be discarded resulting in branch delay.
- Delayed Branching (to be introduced next) is a method that ensures no instructions are discarded after the branch. That means that delay slot instructions are always executed.



- Branch delay reduces pipeline efficiency as delay slot instructions are discarded.
- To reclaim back this efficiency, delayed branching can be used.
 - When delayed branching is enabled, the processor will not discard the delay slot instructions.
- But if the original order of instructions is used with delayed branching, the program execution will be wrong. In other word, we resolve the pipeline inefficiency issue by getting the processor to execute delay slot instructions, but in doing so, we **break the logic flow of the original program**.
- We will illustrate this point again in the next slide.

Delayed Branching: Disabled

- For simplification sake, we can view Delayed branching as a feature/choice in the processor
- If Delayed branching is disabled, the processor pipeline will discard the delay slot instructions if the branch is True to preserve the correctness of the program logic, at the expense of cycle wastage.

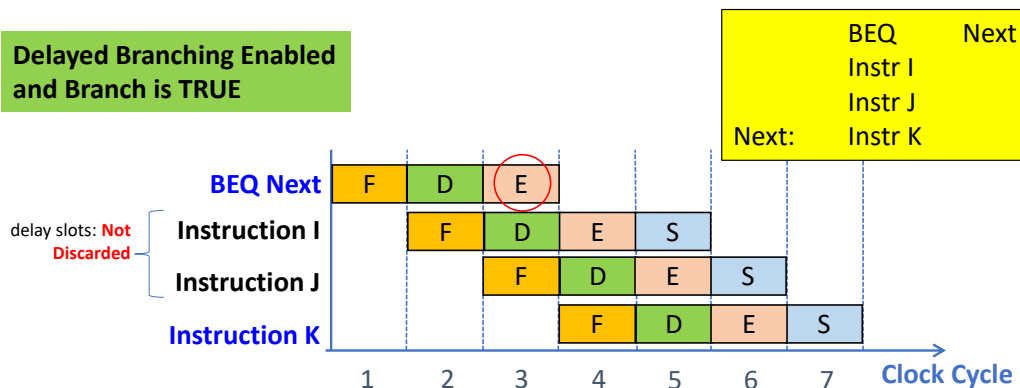


- The pipeline diagram here illustrate the case where Delayed branching is disabled, i.e. delay slot instructions are discarded when branch is true.
- The instructions are discarded to maintain the proper program logic, i.e. after BEQ, instruction K will be executed. I and J should not be executed.

Delayed Branching: Enabled

- If the Delayed branching is **enabled**, the processor will **execute** the **delay slots instructions** regardless of true or false branch decision so **no cycles are wasted**.
- However, **the program logic would be wrong** if instruction **I** and **J** are executed, when the branch is True.
- So, **some other instructions** (other than I and J) need to fill the delay slots.

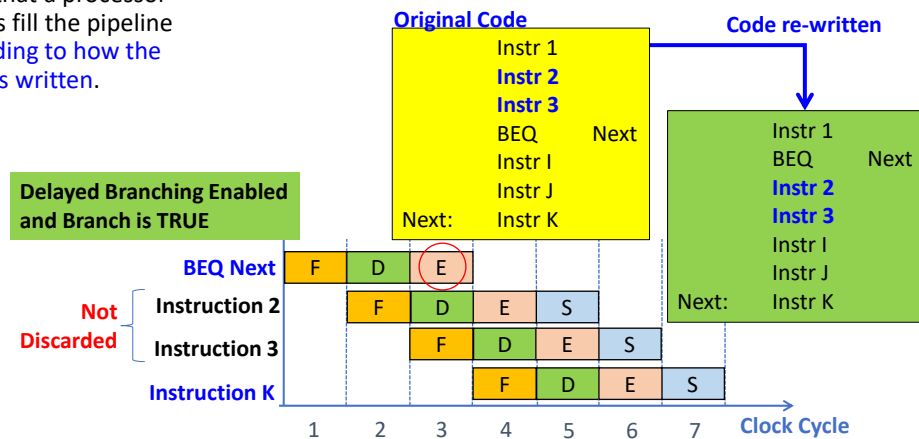
Delayed Branching Enabled
and Branch is TRUE



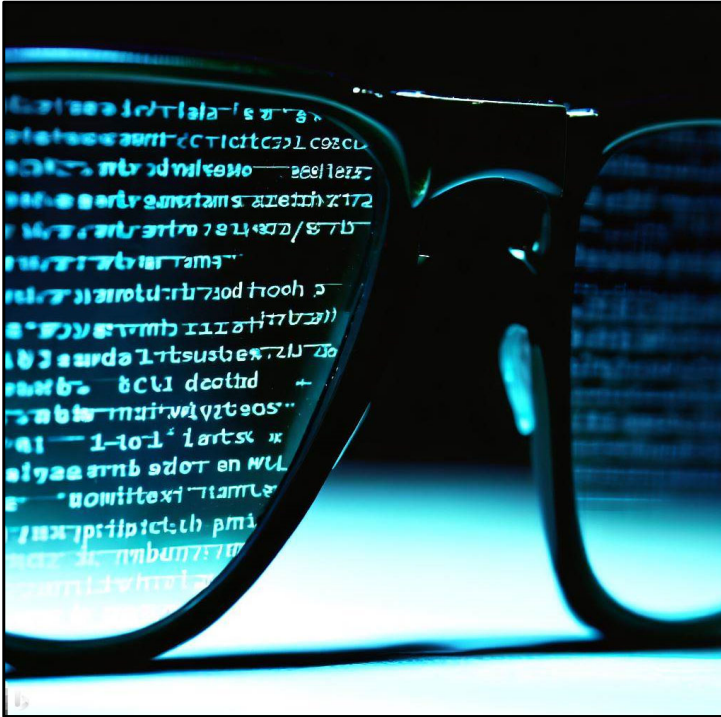
- With delayed branching enabled, the processor will execute delay slot instructions to completion.
- If we kept everything as per the previous slide, that means instruction I and J will be executed even when branch is true, which is obviously wrong from program logic point of view.
- That means there is a need to re-arrange the program code so that some other instructions will get loaded into the delay slots instead of I and J, and yet allow a proper program logic flow.
- Note again that processor will load the pipeline according to how the code is written.

Delayed Branching

- The user or compiler needs to fill the delay slots with Independent instructions, by changing the program.
- For example, if instructions 2 and 3 are independent instructions, they can be made to occupy the delay slots by re-writing the program code.
- Note that a processor always fill the pipeline according to how the code is written.



- If we expand the code a little to include more instructions, we have instructions 1, 2, 3 followed by BEQ followed by instructions I, J, K. As shown in the yellow box.
- This code needs to be **re-written** in order to use the delayed branching feature.
- The re-written code is shown in the green box. You can see that we have brought two instructions to below BEQ.
 - So would this code behave similar to the original program logic flow?
- Let's go back to the yellow box which has the original code. The program logic will have two paths corresponding to the cases where branch is true and false.
 - If branch is true, execution sequence will be instructions 1, 2, 3, BEQ and K.
 - If branch is false, execution sequence will be instructions 1, 2, 3, BEQ and I
- Moving over to the green box and with delayed branching enabled,
 - If branch is true, execution sequence will be instructions 1, BEQ, 2, 3 and K
 - If branch is false, execution sequence will be instructions 1, BEQ, 2, 3 and I
- You can see that other than the execution sequence of instructions 2 and 3, basically the same group of instructions are executed for the case of true and false branch.
 - The fact that instructions 2 and 3 are executed after BEQ means that instructions 2 and 3 have to be independent instructions that does not affect the branch decision making process.



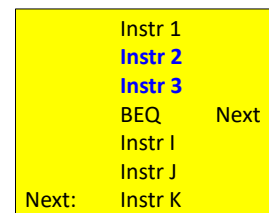
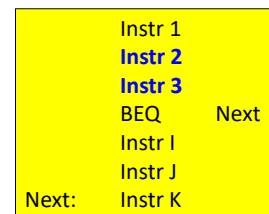
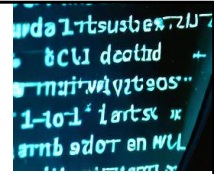
Delayed Branching

- User or Compiler can schedule independent instructions to be filled in the delay slots after the branch.
- If such independent instruction exists, they will always be executed, leading to zero branch delay, since no cycles are wasted.
- If an independent instruction cannot be found or if there are insufficient number of independent instructions to fill the delay slots, NOP instruction(s) should be used to populate the delay slots to preserve the correctness of the original program logics.

- As discussed in the previous slide, when delayed branching is enabled, delay slots has to be filled with independent instructions that doesn't affect the branch decision making process.
- If such independent instructions cannot be found, then the delay slot has to be filled with NOP instruction.

Delay slot Instructions

- Needs to be Independent instructions which **does not play a part in the branch decision making process**.
- Another requirement for delay slot instructions is that they will **always be executed in the original program flow**, regardless of whether the branch is taken or not.
- There are a few **general rules of thumb**
- It should be some instructions that are **earlier sequence wise** in the original program, compared to the branch instruction. Any instruction that occurs later than the branch instruction in the original program would be executed or not executed depending on the branch decision.
- Its operation **should not have effect on status of any registers that would in turn affect the branch decision**. e.g. if a BEQ is used, the independent instruction should not affect the Z flag which BEQ instruction used for decision making.
- This is because **delay slot instructions gets fully executed only after the branch decision had been made**, which means delay slots instruction had **no effect on the branch decision**.
- If the Branch is **part of a loop**, delay slot instructions needs to be **sourced from instructions within the loop** as delay slot instruction will go through the same number of iterations as well.



- Some details on delay slots instruction selection.
- These are instructions that do not affect the branch decision. On top of that, these instructions will always be executed regardless of the outcome of the branch decision.
- Some rule of thumb in choosing delay slot instructions
 - They should be instructions that are earlier in the sequence of the program code, before the Branch instruction.
 - Instructions which are after the branch depends on the branch decision and may not always be executed.
 - Delay slot instructions should not affect the branch decision making process. This is because when they are brought over to occupy the delay slots, they will be executed after the branch decision has been made, which means that they will not be able to influence the branch decision.
 - E.g. if a BEQ is used, and instruction 3 affect the Z flag used in the BEQ instruction. This will be ok in the original program code since instruction 3 is executed before BEQ. However, if delayed branching is enabled and instruction 3 is selected to occupy the delay slots, it will be executed after BEQ and will no longer be able to affect the BEQ decision making process, which is wrong from program logic

perspective.

Dynamic Branch Prediction:

another way to mitigate branch delay

- Hardware circuitry to guess outcome of a conditional branch
 - **Branch history table is implemented to store the predicted target addresses of branch instructions in the program**
- If prediction is **correct**
 - **Continue normal execution – no wasted cycles**
- If prediction is **incorrect**
 - **Flush instructions that were incorrectly fetched – wasted cycles**
 - **Update prediction bit and target address for future use**



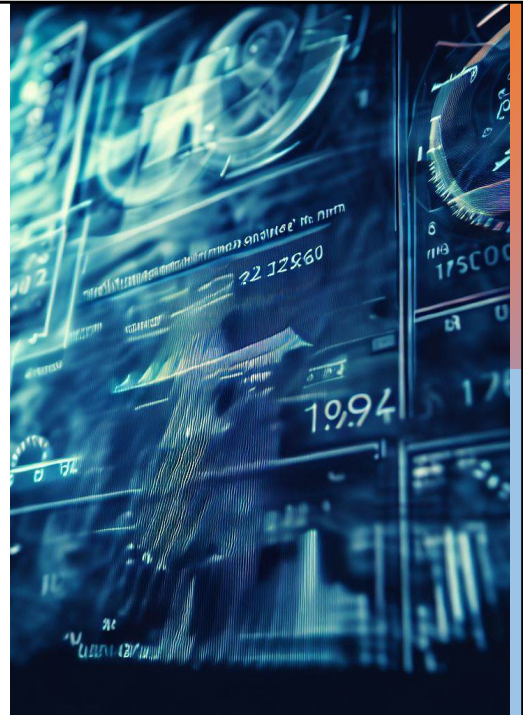
Address of Branch Instruction	Predicted Target address	Prediction Result (T/F)
Address of "BEQ Next"	Next	T



- One other way to mitigate branch delay is to employ dynamic branch prediction.
- There are many different prediction algorithms
- For example, the processor can maintain a branch history table to store the predicted target address of various branch instructions in the program.
- The processor will load the instructions according to the prediction,
 - e.g. using code in the yellow box, if it predicts that BEQ is true, then it'll fetch instruction K after BEQ and not instruction I.
 - If it predicts BEQ to be false, it will load instruction I instead of instruction K.
- If the prediction is correct, no cycles will be wasted.
- If prediction is incorrect, the same flushing of pipeline will occur and similar cycle wastage as the branch delay will be incurred.
- It is actually not too difficult to have a fairly effective branch prediction algorithm due to the nature of a typical program code.

Connecting to the Real World

- ARM Cortex M3/M4 Processor
 - 3-stage pipeline. Instruction Fetch, Instruction Decode and Instruction Execute)
- Branch speculation.
 - When a branch instruction is encountered, the decode stage also includes a speculative instruction fetch that could lead to faster execution.
 - The processor fetches the branch destination instruction during the decode stage itself.
 - During the execute stage, the branch is resolved and it is known which instruction is to be executed next.
 - If the branch is not taken, the next sequential instruction is already available.
 - If the branch is taken, the branch instruction is made available at the same time as the decision is made.



- The pipeline architecture we discussed here is not the one in the ARM processor.
- Different ARM processor sub-families have different types of pipeline architectures; the one used in the MSP432 ARM processor you encounter in your Lab4 has a 3-stage pipeline: Fetch, Decode and Execute.
- On top of that, it has a branch speculation feature that fetches the instruction in the case of true branch of the program flow; i.e., both instructions are available in CPU for having branch or no branch; one of them will be used depending on the branch decision. In other words, no branch delay incurred.
- That concludes our discussion on the chapter of pipeline processors. You will revisit this again in your Advanced Comp Arch Module.

No part of this material shall be filmed, recorded, downloaded, reproduced, distributed, republished, or transmitted in any form or by any means without written approval from Nanyang Technological University.